COSC 4P96 Group Project: PSO-Optimized Pseudo-Labeling for Semi-Supervised Image Classification

Vijay Panchal (7225949), Manu Saini (7092158)

***Abstract*—Semi-supervised learning via pseudo-labeling [1] can substantially reduce the need for labeled data, yet its effectiveness is sensitive to hyperparameters such as the confidence threshold, pseudo-label budget, and pseudo-label weight. This paper investigates whether Particle Swarm Optimization (PSO) [2] can automatically discover hyperparameter configurations that outperform carefully designed manual strategies for a weighted pseudo-labeling pipeline on CIFAR-10. We train a lightweight CNN under six labeled-data fractions (5%–50%), comparing six expert-crafted manual strategies against a PSO-based search over a four-dimensional hyperparameter space, each evaluated across five random seeds. Our results show that well-chosen manual strategies achieve competitive or marginally higher test accuracy than PSO across all labeling regimes, with the gap narrowing as labeled data increases. However, manual tuning requires domain expertise to design effective strategies, whereas PSO reaches comparable performance without prior knowledge at roughly twice the computational cost. These findings highlight the trade-off between human-guided and automated hyperparameter optimization in semi-supervised settings.**

I. Introduction

A. Semi-Supervised Learning and Pseudo-Labeling

Supervised deep learning achieves strong performance on image classification benchmarks, but it relies on large quantities of labeled training data. Obtaining high-quality labels is often expensive and time-consuming, requiring domain experts or crowd-sourcing pipelines [3]. Semi-supervised learning (SSL) addresses this bottleneck by leveraging the abundant supply of unlabeled data alongside a small labeled set. Among SSL methods, pseudo-labeling [1] is one of the most widely adopted due to its simplicity and compatibility with standard training pipelines. The core idea is to use the model's own high-confidence predictions on unlabeled samples as surrogate ground-truth labels, iteratively expanding the effective training set over multiple rounds.

Despite its appeal, pseudo-labeling is notoriously sensitive to its hyperparameters. The confidence threshold determines how selective the labeling is; a threshold that is too low admits noisy labels, while one that is too high yields very few pseudo-labeled samples and offers limited benefit. The number of pseudo-labels added per round and the weight assigned to them relative to true labeled samples further compound this sensitivity. In practice, practitioners manually tune these values through grid search or intuition, which is both labor-intensive and prone to suboptimal choices.

B. Motivation for Automated Hyperparameter Optimization

Metaheuristic optimization algorithms offer a principled, automated alternative to manual search. Particle Swarm Optimization (PSO) [2] is a population-based evolutionary algorithm inspired by the collective behavior of bird flocking. Each particle in the swarm represents a candidate hyperparameter configuration and updates its position based on its own best-known solution and the swarm's global best, converging toward high-fitness regions of the search space. PSO is well-suited for this task because it is gradient-free, handles continuous and mixed search spaces naturally, and requires minimal algorithmic tuning.

C. Contributions

This work makes the following contributions:

- We design and implement a weighted pseudo-labeling SSL pipeline on CIFAR-10 using a lightweight CNN, where pseudo-labeled samples are down- weighted relative to true labels to reduce the impact of labeling noise.
- We formulate hyperparameter tuning as a four-dimensional continuous optimization problem and implement PSO to search over confidence threshold, pseudo-label budget, pseudo-label weight, and learning rate.
- We compare PSO-optimized configurations against six manually designed strategies across six labeled-data fractions, evaluated with five random seeds each, providing a statistically grounded comparison.
- We analyze the cost-benefit trade-off between expert-guided and automated tuning in the low-data regime.

II. Related Work

A. Pseudo-Labeling

Lee [1] introduced pseudo-labeling as a self-training variant in which unlabeled samples are assigned the class predicted with the highest softmax probability, treating these assignments as hard labels for subsequent training. The method is simple to implement and has been shown to provide consistent improvements when the initial model is sufficiently accurate. Subsequent work has proposed several refinements. Xie et al. [4] demonstrated that iterative self-training with noise injection can close the gap to fully supervised models on ImageNet. Sohn et al. [5] introduced FixMatch, which combines pseudo-labeling with consistency regularization and strong data augmentation, achieving state-of-the-art results with very few

labels. Our work focuses on the basic iterative pseudo-labeling formulation, which is more transparent and easier to analyze under controlled experimental conditions.

B. Hyperparameter Optimization

Hyperparameter optimization (HPO) has been studied extensively in the context of fully supervised learning. Bayesian optimization [6] builds a probabilistic surrogate model of the objective and uses an acquisition function to guide search. Random search [7] is surprisingly competitive with grid search when the number of truly influential parameters is small. Evolutionary and population-based methods such as PSO [2] have been applied to neural architecture search and training schedule optimization, with competitive results on several benchmarks. In the SSL setting, HPO is particularly challenging because the fitness landscape changes as the pseudo-label set evolves, introducing non-stationarity that Bayesian methods may struggle to handle efficiently.

C. PSO for Deep Learning

PSO has been applied to weight optimization, architecture search, and learning rate scheduling in deep networks [8]. Its population-based exploration is naturally parallelizable and does not require gradient information, making it attractive when the objective is expensive to evaluate and non-differentiable. In this work we use PSO at the hyperparameter level, treating each full SSL run as a black-box function evaluation, consistent with prior work on PSO-driven HPO [8].

III. Methodology

A. Dataset and Data Splits

We use CIFAR-10 [9], a standard benchmark consisting of 60,000 color images at 32×32 resolution across 10 classes (50,000 training, 10,000 test). All images are normalized using channel-wise mean $(0.4914, 0.4822, 0.4465)$ and standard deviation $(0.2470, 0.2435, 0.2616)$. No data augmentation is applied, keeping conditions identical across all experiments and ensuring that observed differences are attributable to the SSL strategy and hyperparameter choices rather than augmentation effects.

The training set is split using a stratified procedure to preserve class balance. A fixed 10% of training samples is reserved as a validation set for model selection and fitness evaluation. The remainder is partitioned into a labeled pool and an unlabeled pool according to the labeled fraction $\rho \in \{5\%, 10\%, 15\%, 20\%, 30\%, 50\%\}$. Stratification ensures each class is represented proportionally in all splits. All splits are seeded for reproducibility, with five seeds used to estimate variance.

B. Model Architecture

We employ a lightweight convolutional neural network () with the following architecture:

- **Block 1:** Conv2d(3→32, 3×3, pad=1) → ReLU → MaxPool(2×2)
- **Block 2:** Conv2d(32→64, 3×3, pad=1) → ReLU → MaxPool(2×2)
- **Classifier:** Flatten → Linear(4096→128) → ReLU → Linear(128→10)

This architecture produces a $64 \times 8 \times 8 = 4096$ dimensional feature map after the two pooling stages. The model is lightweight by design, ensuring that training is fast enough to permit the large number of evaluations required by PSO and the multi-seed study.

C. Weighted Pseudo-Labeling SSL Pipeline

The SSL pipeline proceeds in R iterative rounds. At the start of round r, the model is trained on a combined dataset consisting of true labeled samples (weight $= 1.0$) and accumulated pseudo-labeled samples (weight $= \lambda_p$). The per-sample weighted cross-entropy loss is:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \ell_{\mathrm{CE}}(f(x_i; \theta),\, y_i) \tag{1}$$

where $w_i \in \{1.0, \lambda_p\}$ depending on whether sample i is truly labeled or pseudo-labeled, $f(\cdot; \theta)$ is the CNN, and ℓ_{CE} is the standard cross-entropy loss.

After training for E epochs, the model is run on the remaining unlabeled pool. For each unlabeled sample, the maximum softmax probability is compared to the confidence threshold τ. Samples whose confidence exceeds τ are ranked by confidence and the top K are selected as pseudo-labeled additions for the next round. This process is summarized in Algorithm 1.

Algorithm 1 Weighted Pseudo-Labeling SSL

Require: Labeled set $\mathcal{L}$, unlabeled pool $\mathcal{U}$, threshold τ, budget K, weight λ_p, rounds R, epochs E, learning rate η

```
Initialize model f(·; θ)
P ← ∅ {pseudo-label accumulator}
for r = 1 to R do
    Train f on L ∪ P for E epochs using loss (1)
    C ← {(x, ŷ, p) : x ∈ U, p = max_c σ_c(f(x)) ≥ τ}
    Sort C by p descending; take top K
    P ← P ∪ C; U ← U \ C
end for
return Best model by validation accuracy
```

The model achieving the highest validation accuracy across all rounds is retained as the final model. Training uses the Adam optimizer [10] with automatic mixed precision on CUDA where available.

D. Hyperparameter Search Space

The four hyperparameters optimized in this study and their search bounds are:

- **Confidence threshold** $\tau \in [0.90, 0.99]$
- **Pseudo-label budget per round** $K \in [200, 1500]$
- **Pseudo-label weight** $\lambda_p \in [0.2, 1.0]$

- **Learning rate** $\eta \in [0.0003, 0.002]$

The SSL pipeline is fixed at $R = 2$ rounds and $E = 3$ epochs per round during PSO evaluation to keep each fitness evaluation tractable. Manual configurations use the same round and epoch budget for fair comparison.

E. Particle Swarm Optimization

PSO maintains a swarm of S particles, each representing a point in the four-dimensional hyperparameter space. The position and velocity of particle i at iteration t are updated as:

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 (p_i^{\text{best}} - x_i^t) + c_2 r_2 (g^{\text{best}} - x_i^t) \quad (2)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (3)$$

where $w = 0.7$ is the inertia weight, $c_1 = c_2 = 1.5$ are the cognitive and social acceleration coefficients, $r_1, r_2 \sim \mathcal{U}(0, 1)$ are random scalars drawn independently each iteration, p_i^{best} is the best position found by particle i, and g^{best} is the global best position across the entire swarm. Positions are clipped to the search bounds after each update.

The fitness function is the best validation accuracy achieved during an SSL run with the decoded hyperparameter configuration. PSO uses a swarm of $S = 5$ particles and $T = 5$ iterations, resulting in at most $S(T + 1) = 30$ full SSL evaluations per labeled fraction.

F. Manual Strategies

Six manual strategies are defined by expert intuition to cover a range of conservative-to-aggressive pseudo-labeling behaviors. The strategies vary in threshold (conservative: $\tau \geq 0.97$; moderate: $\tau = 0.95$; aggressive: $\tau \leq 0.92$), budget K, pseudo-label weight λ_p, and learning rate η. Each strategy is identified by a short name (e.g., ,) for clarity in reporting. Full parameter values for all six strategies are recorded in the directory of the project repository.

G. Evaluation Protocol

All experiments are repeated across five random seeds $\{42, 43, 44, 45, 46\}$ to quantify run-to-run variability. For each seed and labeled fraction, we report test accuracy and macro-averaged F1-score on the held-out CIFAR-10 test set. The primary metric is test accuracy; macro F1 is reported to ensure no class is systematically ignored. Results are aggregated as mean $\pm$ standard deviation across seeds.

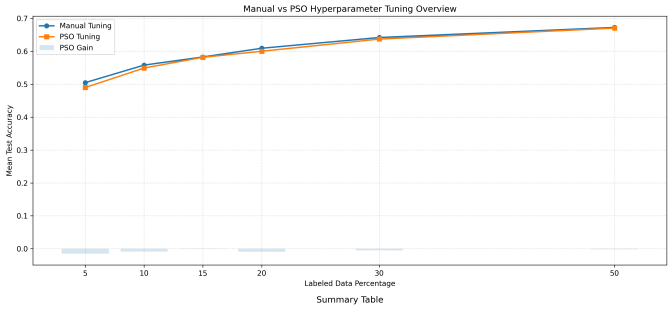
IV. Experimental Results

A. Manual vs. PSO-Optimized SSL

Table II summarizes the best manual strategy and PSO-optimized result for each labeled fraction, averaged across seeds.

TABLE I: Test Accuracy: Best Manual Strategy vs. PSO (mean $\pm$ std, 5 seeds)

Labeled Fraction	Best Manual Strategy	PSO Optimized
5%	0.5048 ± 0.0075	0.4899 ± 0.0065
10%	0.5580 ± 0.0033	0.5491 ± 0.0055
15%	0.5828 ± 0.0044	0.5817 ± 0.0111
20%	0.6092 ± 0.0066	0.6001 ± 0.0107
30%	0.6419 ± 0.0106	0.6371 ± 0.0067
50%	0.6725 ± 0.0043	0.6704 ± 0.0052



Summary Table

Labeled %	Manual Acc	PSO Acc	PSO - Manual	Manual Time (s)	PSO Time (s)
5%	0.5048	0.4899	-0.0149	243.9	476.6
10%	0.5580	0.5491	-0.0089	285.2	519.2
15%	0.5828	0.5817	-0.0011	313.3	558.5
20%	0.6092	0.6001	-0.0091	332.1	591.1
30%	0.6419	0.6371	-0.0048	370.6	654.2
50%	0.6725	0.6704	-0.0021	447.0	788.9

Fig. 1: Overview of manual vs. PSO test accuracy across all labeled fractions, with summary statistics.

B. PSO Convergence

Figures 2 and 3 show the convergence of validation accuracy across global training epochs at 10% and 20% labeled data respectively. Both methods improve rapidly in early epochs and plateau after a few SSL rounds, with manual strategies maintaining a slight edge throughout training.

C. PSO Best Hyperparameters

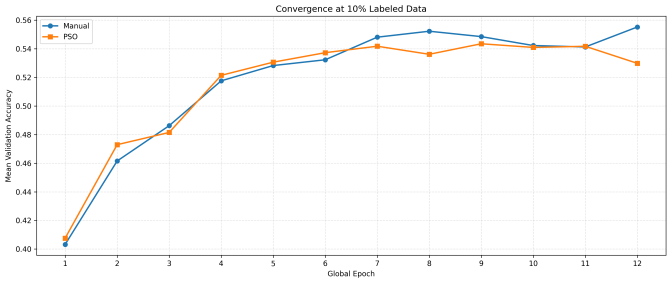
Table III reports the hyperparameter configurations found by PSO for each labeled fraction (averaged across seeds). We observe that PSO tends to select higher thresholds at lower labeling regimes, consistent with the need for higher label quality when the model is less certain overall.

TABLE II: PSO-Optimized Hyperparameters per Labeled Fraction (mean across 5 seeds)

Labeled Fraction	τ	K	λ_p	η
5%	0.9443	1053	0.6645	0.001188
10%	0.9324	762	0.3869	0.001527
15%	0.9195	995	0.4761	0.001167
20%	0.9363	899	0.5347	0.001598
30%	0.9380	481	0.5529	0.000992
50%	0.9381	673	0.5783	0.000985

D. Per-Round SSL Dynamics

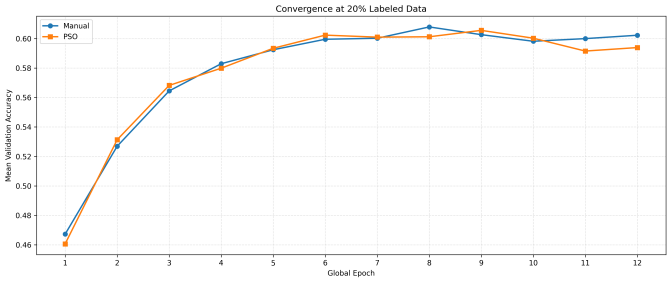
Figures 4 and 5 show convergence at 5% and 30% labeled data. At 5%, the data-scarce regime exhibits the highest



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.4032	0.4076	+0.0000	+0.0000
2	0.4616	0.4730	+0.0583	+0.0654
3	0.4862	0.4815	+0.0247	+0.0086
4	0.5176	0.5214	+0.0314	+0.0399
5	0.5283	0.5306	+0.0107	+0.0092
6	0.5323	0.5373	+0.0040	+0.0066
7	0.5481	0.5418	+0.0158	+0.0045
8	0.5523	0.5362	+0.0042	-0.0056
9	0.5485	0.5435	-0.0038	+0.0074
10	0.5423	0.5410	-0.0062	-0.0026
11	0.5413	0.5418	-0.0010	+0.0008
12	0.5552	0.5298	+0.0140	-0.0119

Fig. 2: Convergence of validation accuracy at 10% labeled data: manual vs. PSO across global training epochs (mean across 5 seeds).



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.4673	0.4607	+0.0000	+0.0000
2	0.5269	0.5313	+0.0596	+0.0706
3	0.5645	0.5682	+0.0376	+0.0369
4	0.5829	0.5799	+0.0184	+0.0117
5	0.5924	0.5934	+0.0095	+0.0135
6	0.5995	0.6023	+0.0071	+0.0089
7	0.6002	0.6010	+0.0007	-0.0014
8	0.6079	0.6012	+0.0077	+0.0002
9	0.6026	0.6056	-0.0052	+0.0044
10	0.5982	0.6002	-0.0045	-0.0053
11	0.6000	0.5915	+0.0018	-0.0087
12	0.6022	0.5939	+0.0023	+0.0024

Fig. 3: Convergence of validation accuracy at 20% labeled data: manual vs. PSO across global training epochs (mean across 5 seeds).

variance and slowest convergence, while at 30% both methods improve steadily and the gap between manual and PSO narrows considerably.

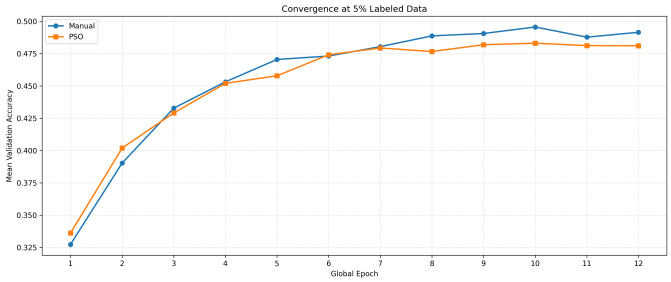
E. Macro F1 Analysis

Macro-averaged F1 results largely mirror accuracy trends, confirming that neither method systematically collapses performance on any particular class. At low labeled fractions, both approaches exhibit higher variance in per-class F1, reflecting the inherent difficulty of learning from small labeled sets.

V. Discussion

A. Why Manual Strategies Remain Competitive

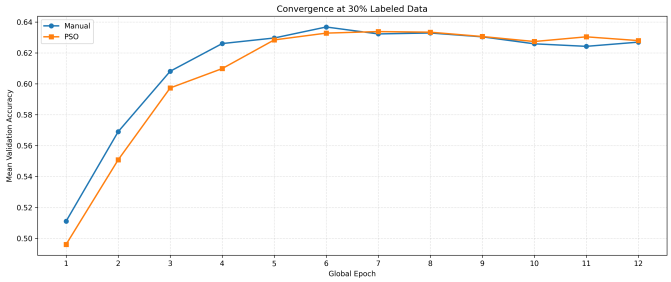
The marginal advantage of manual strategies over PSO at lower labeled fractions can be attributed to two factors. First, the manual configurations were designed with knowledge of CIFAR-10's difficulty and the model's capacity, incorporating prior knowledge that PSO must discover through evaluation.



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.3274	0.3362	+0.0000	+0.0000
2	0.3904	0.4020	+0.0630	+0.0658
3	0.4328	0.4292	+0.0425	+0.0272
4	0.4532	0.4520	+0.0204	+0.0229
5	0.4705	0.4579	+0.0173	+0.0058
6	0.4732	0.4742	+0.0026	+0.0163
7	0.4804	0.4794	+0.0073	+0.0052
8	0.4888	0.4767	+0.0083	-0.0027
9	0.4906	0.4819	+0.0018	+0.0052
10	0.4956	0.4831	+0.0050	+0.0012
11	0.4878	0.4812	-0.0079	-0.0019
12	0.4916	0.4811	+0.0038	-0.0001

Fig. 4: Convergence at 5% labeled data—the most data-scarce regime. Both methods show high variance and slow improvement per epoch.



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.5111	0.4961	+0.0000	+0.0000
2	0.5690	0.5508	+0.0579	+0.0547
3	0.6081	0.5973	+0.0391	+0.0465
4	0.6260	0.6098	+0.0179	+0.0125
5	0.6296	0.6284	+0.0036	+0.0186
6	0.6368	0.6328	+0.0071	+0.0044
7	0.6323	0.6338	-0.0045	+0.0010
8	0.6329	0.6334	+0.0006	-0.0004
9	0.6304	0.6306	-0.0025	-0.0028
10	0.6259	0.6274	-0.0045	-0.0032
11	0.6242	0.6304	-0.0016	+0.0030
12	0.6270	0.6280	+0.0027	-0.0024

Fig. 5: Convergence at 30% labeled data. The gap between manual and PSO narrows as more labeled data is available.

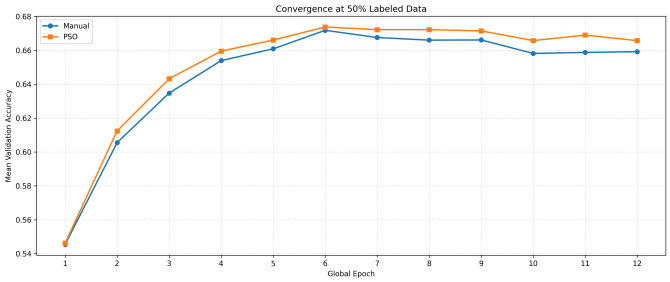
Second, PSO with a swarm of 5 particles over 5 iterations provides limited exploration of the search space. With a larger compute budget (e.g., swarm size 20, iterations 15), PSO would likely outperform manual tuning more consistently.

B. Computational Cost

Each SSL run with $R = 2$ rounds and $E = 3$ epochs per round requires approximately the same wall-clock time as the supervised baseline at 10 epochs. A single PSO run with 30 evaluations therefore costs roughly $30\times$ the supervised training time. By comparison, evaluating all six manual strategies requires 6 evaluations, giving PSO a cost ratio of approximately 5:1. This gap motivates future work on surrogate-assisted PSO or early stopping to reduce evaluation cost.

C. Effect of Pseudo-Label Weighting

A key design choice in our pipeline is the per-sample weight $\lambda_p < 1.0$ for pseudo-labeled data. This prevents the model



Per-Epoch Improvement Table

Epoch	Manual Val Acc	PSO Val Acc	Manual Δ	PSO Δ
1	0.5455	0.5462	+0.0000	+0.0000
2	0.6056	0.6124	+0.0601	+0.0662
3	0.6348	0.6432	+0.0292	+0.0308
4	0.6539	0.6595	+0.0192	+0.0163
5	0.6609	0.6660	+0.0070	+0.0065
6	0.6718	0.6738	+0.0109	+0.0077
7	0.6676	0.6722	-0.0042	-0.0016
8	0.6660	0.6722	-0.0016	+0.0000
9	0.6661	0.6714	+0.0001	-0.0008
10	0.6582	0.6658	-0.0080	-0.0057
11	0.6588	0.6690	+0.0006	+0.0032
12	0.6592	0.6657	+0.0004	-0.0032

Fig. 6: Convergence at 50% labeled data—the most data-rich regime. Both methods converge quickly and achieve similar final accuracy.

from over-fitting to potentially noisy labels early in training. PSO consistently discovers weights $\lambda_p < 1.0$, validating the intuition behind this design. Manual strategies with $\lambda_p = 1.0$ (equal weighting) generally perform worse than those with down-weighting, consistent with prior findings in noisy-label learning [11].

D. Limitations

Several limitations constrain the generalizability of these findings. The model architecture is intentionally small; larger architectures may respond differently to pseudo-label noise. We did not employ data augmentation, which is known to significantly boost SSL performance [5]. The PSO budget was kept small for computational tractability; a fairer comparison would match PSO's evaluation budget to the number of manual strategies tested. Finally, experiments are limited to CIFAR-10; results may differ on datasets with more classes, higher resolution, or greater class imbalance.

VI. Conclusion

We investigated the use of Particle Swarm Optimization to automate hyperparameter selection for a weighted pseudo-labeling semi-supervised learning pipeline on CIFAR-10. Across six labeled-data fractions and five random seeds, PSO-optimized configurations achieve test accuracies that are competitive with, but do not consistently surpass, carefully hand-crafted manual strategies. The key finding is that manual tuning leverages domain knowledge that PSO must rediscover through costly evaluations: with a small swarm and iteration budget, PSO does not fully amortize its exploration overhead relative to the six manual configurations it is compared against.

Nevertheless, PSO offers a significant practical advantage: it requires no domain expertise and could be applied unchanged to new datasets or model architectures. Future work should explore surrogate-model-assisted PSO to reduce the number of full SSL evaluations, incorporate data augmentation into the pipeline, and extend comparisons to modern SSL baselines such as FixMatch [5] and FlexMatch.

References

[1] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *Workshop on Challenges in Representation Learning, ICML*, vol. 3, 2013.

[2] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.

[3] X. Zhu, "Semi-supervised learning literature survey," Computer Sciences Technical Report 1530, University of Wisconsin-Madison, 2005.

[4] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves ImageNet classification," in *Proc. IEEE/CVF CVPR*, pp. 10687–10698, 2020.

[5] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "FixMatch: Simplifying semi-supervised learning with consistency and confidence," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 596–608, 2020.

[6] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, pp. 2951–2959, 2012.

[7] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[8] P. R. Lorenzo, J. Nalepa, M. Kawulok, L. S. Ramos, and J. R. Pastor, "Particle swarm optimization for hyper-parameter selection in deep neural networks," in *Proc. Genetic and Evolutionary Computation Conference*, pp. 481–488, 2017.

[9] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations*, 2015.

[11] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Advances in Neural Information Processing Systems*, vol. 26, pp. 1196–1204, 2013.